JEREMIAH GEEKS · SAMEDI 31 OCTOBRE 2026 · 12 h – 14 h · SEUL, SUR TON POSTE

🛠️ TP n°2 — Le plateau vivant

Samedi dernier, ton jeu a appris à **se souvenir** : le symbole reste sur le plateau, et le jeu refuse une case déjà prise. Aujourd'hui, **tu travailles seul pendant deux heures** pour que ça devienne solide. À la fin, ton jeu enchaînera quatre coups sans tricher.

Ton nom : | **Ton fichier :** `tp2_tonprenom.py` | **Fini à :** ____ h ____

Comment travailler aujourd'hui. Tu avances seul, à ton rythme. Le professeur passe régulièrement : **il ne fera pas à ta place**, il te fera lire ton erreur. Tu coches chaque exercice réussi. Tu relances (**F5**) après chaque changement, et tu sauvegardes (**Ctrl + S**) après chaque partie. **Bloqué ?** 1) je relis la consigne · 2) je lis le message rouge et je cherche la ligne · 3) je regarde le tableau des erreurs, dernière page. **Après 5 minutes, je passe à la suite** et j'y reviendrai. Les encadrés jaunes ⭐ **ALLEZ PLUS LOIN** sont **facultatifs**.

Partie 0 — Mise en route — 5 min

Ouvre **Thonny** → Fichier → Nouveau → Enregistrer sous `tp2_tonprenom.py` dans `GEEKS / mon_xo`. Écris cette ligne et lance :

```
print("TP2 - Mehdi")
```

☐ **C'est réussi quand** ton prénom s'affiche en bas.

Partie 1 — Échauffement : les listes — 20 min

1.1 — ta liste

Crée une liste `equipe` avec **quatre prénoms** de la classe, puis affiche-la en entier avec `print(equipe)`. ☐

1.2 — un seul élément

Affiche **le premier** prénom, puis **le dernier**. Attention : le premier porte le numéro **0**, pas 1.

```
print(equipe[0])      ← le premier
```

☐ Écris ici le numéro du **dernier** : .....

1.3 — changer un élément

Remplace le deuxième prénom par un autre, puis réaffiche la liste entière. Le changement doit se voir. ☐

1.4 — l'erreur, exprès

Essaie `print(equipe[4])`. Lis le message rouge.

Nom de l'erreur : .....

Ce qu'elle veut dire, avec tes mots : .....

☐

⭐ **ALLEZ PLUS LOIN 1**

Une liste peut contenir des nombres, pas seulement du texte :

```
scores = [10, 25, 7]
print(scores[0] + scores[1])
```

Essaie. Puis fabrique une liste de trois scores, et affiche **leur total**.

Partie 2 — L'indice : la case et son numéro — 20 min

Le point le plus important de la journée. Le joueur parle en **cases** (de 1 à 9), Python compte en **indices** (de 0 à 8).

La case du joueur	1	2	3	4	5	6	7	8	9
L'indice Python	0	1	2	3					

☐ **Complète la deuxième ligne au crayon**, puis écris la formule : indice = ...

2.1 — poser à la main
Crée `cases = ["1", "2", "3", "4", "5", "6", "7", "8", "9"]`, puis, **sans utiliser de formule**, mets un `"X"` dans la case 1, un `"O"` dans la case 5 et un `"X"` dans la case 9. Affiche la liste pour vérifier. ☐

2.2 — poser avec la formule
Maintenant, range `3` dans une boîte `case` et pose un `"O"` en utilisant `case - 1`. Vérifie que le O est bien sur la **troisième** case. ☐

⭐ **ALLEZ PLUS LOIN 2**
Que se passe-t-il avec `cases[-1]` ? Essaie, observe, et écris ce que tu comprends. *(C'est la cause d'un vrai bug de notre jeu : celui du zéro.)*

Partie 3 — Le plateau qui vit

25 min

3.1 — dessiner à partir de la liste
Reprends `plateau` (les neuf numéros) et écris les **cinq lignes** qui dessinent la grille à partir de la liste :

```
print(" " + plateau[0] + " | " + plateau[1] + " | " + plateau[2])
print("---+---+---")
… à toi pour les trois lignes suivantes
```

☐ **C'est réussi quand** la grille affiche 1 à 9, bien alignée.

3.2 — le plateau change
Sous ton affichage, écris `plateau[4] = "X"`, puis **recopie les cinq lignes d'affichage**. Lance : la grille doit s'afficher **deux fois**, et la deuxième a un X au centre. ☐

3.3 — le joueur choisit
Remplace le `4` écrit en dur par une case **demandée au joueur** : trois lignes (demander, transformer en nombre, poser avec `case - 1`). ☐

⭐ **ALLEZ PLUS LOIN 3**
Demande aussi au joueur **quel symbole** il veut poser (par exemple `*` ou `#`), et pose celui-là au lieu de `"X"`.

Partie 4 — LE PROJET : quatre coups, sans triche

35 min

C'est la partie qui compte. Nouveau fichier : `tp2_xo.py`. Ton programme doit enchaîner **quatre coups** — X, O, X, O — et **refuser** une case déjà prise ou un numéro impossible. Voici exactement ce qu'on doit voir (en gras, ce que tape le joueur) :

```
Joueur 1, ton nom ? Joyce
Joueur 2, ton nom ? Gedeon

 1 | 2 | 3
---+---+---
 4 | 5 | 6
---+---+---
 7 | 8 | 9
```

```
Joyce (X), quelle case ? 5

 1 | 2 | 3
---+---+---
 4 | X | 6
---+---+---
 7 | 8 | 9
Gedeon (O), quelle case ? 5
Case deja prise !
… et ainsi de suite jusqu'au 4e coup

Coups reellement joues : 3 sur 4.
Cases encore libres : 6
```

Les cinq étapes :

1	L'accueil, les deux noms, la liste `plateau`, et un compteur `tour = 0`
2	Afficher le plateau, puis demander sa case au joueur 1 et la transformer en nombre
3	Le `if` **dans le** `if` : si la case n'existe pas → refus ; sinon, si elle est libre → on pose et `tour = tour + 1` ; sinon → « Case deja prise ! »
4	**Recopier** les étapes 2 et 3 trois fois : joueur 2 (O), joueur 1 (X), joueur 2 (O)
5	À la fin : le plateau une dernière fois, les coups réellement joués, et les cases libres

☐ **Quand ça marche : Ctrl + S, et lève la main.** Le professeur vient voir ton écran — c'est la validation de la séance.

⭐ **ALLEZ PLUS LOIN 4 — trois défis**

- **Le cinquième coup.** Ajoute un tour de plus. Combien de lignes as-tu dû recopier ?
- **Le compteur de refus.** Une deuxième boîte qui compte les coups impossibles, affichée à la fin.
- **Le plateau au départ.** Essaie de démarrer avec neuf espaces `" "` au lieu des numéros. Qu'est-ce qui ne marche plus ? **Écris pourquoi** — c'est une vraie question d'informaticien.

Partie 5 — La chasse aux bugs 15 min

Cinq programmes cassés. Pour chacun : **écris ce qui ne va pas**, puis corrige dans Thonny.

#	Le code cassé	Ce qui ne va pas
1	`plateau[9] = "X"`	
2	`case = 3` `plateau[case] = "X"`	
3	`plateau = "1", "2", "3"`	
4	`case = input("Case ? ")` `plateau[case - 1] = "X"`	
5	`if plateau[case - 1] = str(case):`	

Bilan — à remplir avant de partir 5 min

Ce que je sais faire maintenant	Tout seul	Pas encore sûr
Créer une liste et afficher un seul élément		
Dire pourquoi la case 1 est à l'indice 0		
Poser un symbole avec `plateau[case - 1]`		
Dessiner le plateau à partir de la liste		
Écrire un `if` dans un `if`		
Reconnaître une `IndexError`		

Si ça ne marche pas — le tableau des erreurs

Ce qui s'affiche	Ce que ça veut dire	Ce que je fais
`IndexError`	« Cette case n'existe pas dans la liste »	Une liste de 9 va de **0 à 8**. Je vérifie mon `- 1`
`TypeError`	Je mélange du texte et un nombre	`int()` pour calculer, `str()` pour coller
`SyntaxError`	« Je ne comprends pas ta phrase »	Guillemets, parenthèses, deux-points. Et `==`, pas `=`
`IndentationError`	« Il faut décaler après les deux-points »	4 espaces. 8 pour le `if` dans le `if`
Le symbole se pose à côté	J'ai oublié le `- 1`	`plateau[case - 1]`, toujours
Le plateau ne change pas	J'affiche **avant** de poser	Je recopie l'affichage **après** la pose